Neural Networks Without Shape Boilerplate: An OCaml DSL Case Study

Łukasz Stafiniak

Independent researcher, sponsored by Ahrefs

Abstract

Modern neural network code is often cluttered with shape bookkeeping: explicit calls to reshape, unsqueeze, expand, transpose, reductions, and integer-axis arguments. OCANNL combines an embedded DSL that removes this boilerplate, with an end-to-end compiler with GPU backends. Users define tensor operations as OCaml functions using concise notation generalizing the Einstein summation convention. Concatenation is an expression over indices e.g. `i^j`, convolution is a contraction with affine operand addressing e.g. `stride*i + dilation*k`. A sequence of axes can be captured by a single variable, e.g. `.. batch..` or `..activations..`, sandwiched between leading and trailing axes; with up to three such variables per shape. We call these *row variables*. The paper showcases three examples: tensor expressions for core multi-head attention and for rank-polymorphic 2D convolution, and a tensor computation for Stochastic Gradient Descent with momentum and Nesterov-inspired correction. OCANNL performs broadcast-aware global bidirectional shape inference and derives loop-index maps for code generation. We formalize the inference problem for the core calculus (excluding affine and concatenation) and show properties of OCANNL's solver (proofs in the appendix). OCANNL's compiler inlines computations to avoid materializing intermediate tensors, performs common subexpression elimination, placement and tiling splits autotune. This paper does not argue for performance — we include preliminary parity-checked benchmarks for transparency, leaving optimization to follow-up work. The presentation corresponds to github.com/ahrefs/ocannl/releases/tag/0.8

Shapes, Tensor Expressions, Syntax Extensions and Examples

OCANNL shapes have three kinds (sequences) of axes, from outermost: batch, output, input. The kind designation is by convention (not enforced, but facilitated). In specifications, the syntax resembles programming language types: `batch | input -> output`. For example: tensor multiplication `*` reduces the input axes of the operator tensor (matrix etc.) with output axes of the operand tensor (vector or matrix etc.), while broadcasting or treating pointwise the batch axes.

A *tensor node* `Tnode.t` is an identity that also carries dimensionality, precision and padding information, but neither data (memory) nor computation. Computations themselves (assignments, type `Assignments.comp`) are expressed in terms of tensor nodes, which makes them hardware (backend) agnostic.

Tensor expressions `Tensor.t` are the primary type of the OCANNL frontend. They combine an inferrable shape, a value node and *forward computation* to update it, and optionally a gradient node and *backprop computation* to propagate the gradient to its component tensors. OCANNL tracks initialization via a field `Tensor.t.params` that's a set of tensors. OCANNL frontend does not have a separate abstraction for machine learning models: any tensor can be considered to be a model, whose trainable parameters are the differentiable subset of its `params`.

OCANNL introduces two syntaxes. Extension point `%op` creates tensor expressions, or OCaml functions returning tensor expressions which we call tensor *operations*. Extension point `%cd` creates tensor computations. Both extensions support inline definitions of new tensors via OCaml's record syntax. For example: `{ w1 = kaiming normal1 () }` inside `%op` introduces tensor `w1` with initialization expression `kaiming normal1 ()`; `{ sgd_momentum }` inside `%cd` below introduces a (non-differentiable, no initialization) tensor `sgd_momentum`;

`{ w_q }` inside `%op` below introduces a tensor with default initialization (e.g. centered uniform distribution).

At the heart of OCANNL are indexing specifications for expressing tensor computations. They generalize the Einstein summation convention: indices missing from the result are reduced over. The specification syntax uses `;` to separate arguments and `=>` to separate out the result. Ellipsis `...` in the specifications are expanded contextually as either `..batch..`, `..input..` or `..output..` row variables. Assignments in the `%cd` syntax specify the unary or binary arithmetic and the accumulation arithmetic explicitly. For the `%op` syntax, we have mixfix operators combining the arithmetic semantics, for example: `++` is identity with additive accumulation, `+*` is multiplication with additive accumulation. These operators also support *variable capture* for both dimensions and rows — the variables from a trailing string list are introduced into scope earlier than they first appear (similarly to inline definitions). The captured variables can be used for both shape constraints `Shape.set_dim`, `Shape.set_equal` and converted to scalars via `dim` (for row variables, the value is the product of the dimensions of the axes).

```
let%op softmax ~spec ?(temperature = 1.0) () =
  let spec = spec ^ " => " ^ replace_alphanum_by_0 spec in
  fun x ->
    let x_scaled = if Float.(temperature <> 1.0) then x /. !.temperature else x in
    let max_vals = x_scaled @^^ spec in
    let exp_vals = exp (x_scaled - max_vals) in
    exp_vals /. (exp_vals ++ spec)

let%op multi_head_attention ~num_heads ~d_k ~d_v () x =
  let q = { w_q } * x in
  let k = { w_k } * x in
  let v = { w_v } * x in
  let scores =
    (q +* k " ... s | h d; ... t | h d => ... s | t -> h" [ "h"; "d" ])
    /. sqrt (dim d)
  in
  Shape.set_dim h num_heads;
  Shape.set_dim d d_k;
  Shape.set_dim e d_v;
  let attn_weights = softmax ~spec:" ... | t -> ..." () scores in
  { w_o } *
    (attn_weights +* v
       " ... s | t -> h; ... t | h e => ... s | h e" [ "e" ])
```

Figure 1. Core multi-head attention in OCANNL. Axes: query position `s`, key/value position `t`, head `h`, key width `d`, and value width `e`. Batch rank, parameter shapes, score shape, contractions, and loop-index maps are inferred. `@^` is infix for `max`, and `@^^` is the `max` reducing equivalent of `++`. `replace_alphanum_by_0` stands for the elided helper that replaces axis identifiers with the fixed index `0` (in the library: `reduce_specified_axes`).

```
let%op conv2d ~label ?(kernel_size = 3) ?(stride = 1)
    ?(use_padding = true) ?out_channels () x =
  Shape.set_dim kh kernel_size;
  Shape.set_dim kw kernel_size;
  Option.iter out_channels ~f:(Shape.set_dim oc);
  x +* { kernel }
       "... | stride*oh + kh, stride*ow + kw, ..ic..;
            kh, kw, ..ic.. -> ..oc..
        => ... | oh, ow, ..oc.."
       [ "kh"; "kw"; "oc" ]
  + { bias = 0. }
```

Figure 2. Rank-polymorphic 2D convolution. The input is addressed at affine spatial positions `stride*oh+kh` and `stride*ow+kw`. The kernel axes `kh` and `kw`, together with the input-

channel row `..ic..`, are contracted. The context row `...` and output-channel row `..oc..` are inferred and may have arbitrary rank. The `use_padding` argument is consumed implicitly by the syntax extension: a convolution index expression written with plain `+` elaborates to read `use_padding` from scope, selecting size-preserving padding (explicitly: `=+`) when true and valid convolution (`<+`) when false.

```
let sgd_one ~learning_rate ?(momentum = 0.0) ?(weight_decay = 0.0)
    ?(nesterov = false) p =
  [%cd
    { sgd_delta } =: p.grad + (!.weight_decay *. p);
    if Float.(momentum > 0.0) then (
      { sgd_momentum } =:
        (!.momentum *. sgd_momentum) + sgd_delta;
      if nesterov then sgd_delta =+ !.momentum *. sgd_momentum
      else sgd_delta =: sgd_momentum);
    p =- learning_rate * sgd_delta ~logic:"."]
```

Figure 3. Stochastic Gradient Descent with momentum. Introduces intermediate state-tracking tensors in a computation. Optional Nesterov-inspired correction.

OCANNL implements coproduct of axes `^`, generalizing concatenation. It is unlocked by the n-ary operator `++^`. We extend the solution space for dimensions with dimension-0, the unit of the coproduct; it is not a valid shape dimension (actual axes never end up at dimension-0). The operator `++^` reduces additively (users can define their own variants).

PyTorch/TensorFlow	OCANNL	Notes
`torch.cat([a, b], dim=0)`	`(a, b) ++^ "x, ...; y, ... => x^y, ..."`	Concatenate tensors
`torch.stack([a, b], dim=0)`	`[a; b]`	Stack with new axis (block tensor syntax)
`x[:n]` (prefix slice)	`x ++^ "a^b => a"`	Extract prefix (size inferred)
`x[n:]` (suffix slice)	`x ++^ "a^b => b"`	Extract suffix (size inferred)
`x[:-3]` (all but last 3)	`x ++^ "a^3 => a"`	Drop last 3 elements

Shapes and Projections: design choices and claims

We say "row" for analogy with type systems, but our shape rows are sequences of axes rather than records. OCANNL doesn't have a "full-blown" parametric polymorphism because we don't generalize inferred shapes into shape schemes, but it does have "poor man's" parametric polymorphism because we generate fresh shape variables for shape constraints derived at tensor operation call sites. Thus OCANNL has two forms of rank polymorphism: structural (subtyping) via broadcasting and parametric via row variables.

The motivation to have left flank axes in shape (row) terms first came from the slice operation — for example when subdividing data into hierarchical minibatches (like in data parallel batched training). This impacts broadcasting: rather than assuming that implicit axes from broadcasting land to the left of the "left flank", we generalize broadcasting to happen in the middle between left and right flanks of already-determined axes of a shape row. This allows uniform handling of broadcast axes and inferred axes in inference (removing a major source of non-determinism).

However, consistently applying the surface row subtype/refinement relation (formally $\preceq$ in the Appendix, derived from the internal marked order $\sqsubseteq$) to the indexing (aka. *einsum*) specifications seen in examples above would be both counter-intuitive and overly restrictive: the broadcasting behavior of the operands would have to match the position of the row variables in the specification. To relax this, we introduce a *flat*

row equivalence relation (formally $\approx$ in the Appendix), it preserves rank but erases the broadcasting position. Except for relaxing broadcast position checking, $C \approx T^{\text{lhs}}$ is stricter than $C \preceq T^{\text{lhs}}$ would be (see Appendix section 3), improving shape propagation through inference.

In the Appendix, we provide formal semantics for the core shape constraints (for simplicity, excluding affine indexing and concatenation), present the shape and projections inference algorithms (where projections represent tensor indexing for tensor assignment loops), and prove their properties. Proofs are available online. In particular, TR Theorem 5.4, TR Theorem 5.9, and TR Proposition 6.2, together state inference termination and soundness. Inference is declarative — the result does not depend on the ordering of the constraints — except at the solver's documented policy commitments (marker placement and deferred word equations, see the Appendix). The shape inference problem is non-principal, and the algorithm is in general incomplete. Should the incompleteness ever manifest in practice, users can provide additional constraints via for example `Shape.infer_equal`, or variable capture and `Shape.set_equal`. Example of incompleteness from TR Example 6.3, where our inference fails although constraints are satisfiable, with α, β coming from leaf tensors and γ result tensor:

$$\Phi_2 = \{3_b \sqsubseteq \alpha,\ 5_b \sqsubseteq \beta,\ \gamma \sqsubseteq \alpha,\ \gamma \sqsubseteq \beta\}$$

Failing here is the more useful outcome, relative to guessing either $\alpha = 1_\emptyset, \gamma = 5_b$ or $\beta = 1_\emptyset, \gamma = 3_b$. Another example stems from the leniency in the semantics of row equivalence. See Remark after Proposition 2.9 in the appendix:

$$\Phi = \{\ \langle\rho\rangle \approx [\,] \cdot \diamond \cdot [3, 5],\quad [3] \cdot \diamond \cdot [9, 5] \preceq \langle\rho\rangle\ \}$$

Solution $\gamma\rho = [3] \cdot \diamond \cdot [5]$ would satisfy both constraints ($[3, 9, 5]$ vs $[3, 1_\emptyset, 5]$). OCANNL fails here by committing to $\gamma\rho = [\,] \cdot \diamond \cdot [3, 5]$.

Dimension basis instead of axis name

OCANNL does not support attaching axis names to tensors, as both the semantics and ergonomics of indexing are sequence based. Instead OCANNL supports attaching bases (arbitrary identifiers) to dimensions. Dimensions of different bases are incompatible even when they're of the same size. 1_b is not broadcastable for any basis b except for $1_\emptyset$ (basis $\emptyset$ is the identifier `bcast_if_1` and is available for explicit use). When not specified, user provided dimensions get basis `default`. This improves shape inference as it ensures data vectors of dimension 1 don't become accidentally broadcastable.

We track the distinction between parameter tensors and other leaf tensors. When an axis of a parameter tensor has nothing constraining it, inference fails with `"You forgot to specify the hidden dimension(s)"`. When it is of a non-parameter tensor, the unconstrained axis closes upward to $1_\emptyset$.

Contexts and explicit compilation

OCANNL has an immutable context `Context.t` that unifies compilation, device, and execution contexts. Root contexts determine the device. Here are the most important operations on a context — the bindings argument of `compile` contains the externally bound indexing symbols used in the computation:

```
val compile : t -> Assignments.comp -> Indexing.unit_bindings -> t * routine
val run : t -> routine -> t
val copy : src:t -> dst:t -> Tnode.t -> unit
val to_host : t -> Tnode.t -> Ndarray.t
val from_host : t -> Tnode.t -> Ndarray.t -> t
```

Contexts track dependencies (e.g. whether a tensor node is initialized before use) both at routine's compile time and at runtime.

OCANNL has multi-stage shape inference. Stage 1 runs as tensor expressions are constructed. Calling `Context.compile` triggers stages 2 and later for constraints that are in flight (global store) at that time.

Benchmarks

We report preliminary results from OCANNL's cross-framework benchmark, which compares OCANNL against PyTorch (eager, and `torch.compile` where available) and tinygrad (JIT, and BEAM search where available) on identical workloads. There are four workloads: `mlp_small` and `mlp_wide` train n-layer relu MLPs with softmax cross-entropy and plain SGD (overhead- vs GEMM-dominated); `lenet` trains LeNet-5 with valid convolutions; `gpt2_mini` runs a pre-LN GPT-2-style decoder (4 layers, model width 256, 8 heads, sequence length 128) forward-only.

The per-machine result tables are collected in the Benchmark tables appendix. The snapshot they give: on CPU (`cc` backend) OCANNL is competitive with the reference frameworks' CPU cells on the MLP and LeNet workloads, while the GPU backends currently trail the mature GPU stacks — by one to two orders of magnitude on the attention workload — since kernel scheduling (fusion granularity, memory-hierarchy placement) is still early-stage work; the Conclusions sketch the planned search-based direction.

Related work

OCANNL's shape inference design arose from the author's appreciation for the readability of einsum notation in JAX, and familiarity with constraint-based type inference combining parametricity and subtyping. In particular, variable elimination, such as the closure computation in *Pottier, "A Framework for Type Inference with Subtyping", ICFP 1998*, inspired OCANNL's shape inference solver. Other related works discussed below are not influences; we only offer a selection as an exhaustive treatment would require a survey paper.

Dependent types enable tracking tensor shapes in tensor types; they are included in restricted forms in ergonomic programming languages with type inference. The DML system *Xi, Pfenning, "Dependent Types in Practical Programming", 1999* paved the way to GADTs and refinement types. The most prominent current example is Futhark *Bailly, Henriksen, Elsman, "Shape-Constrained Array Programming with Size-Dependent Types", 2023*. It has capable type inference, but no rank-polymorphism. This contrasts with Qube *Trojahner, Grelck, "Dependently typed array programs don't go wrong", 2009*. Qube has parametric rank-polymorphism but no type inference.

An interesting recent work is Star *Bachurski, Mycroft, and Orchard, "Structuring Arrays with Algebraic Shapes", 2025*. It interprets algebraic data types as types of tensor indices. This is reminiscent of OCANNL's connection between shapes and projections. Star supports structural rank-polymorphism via subtyping: as in OCANNL, Star's shapes form a lattice (but as in type systems, the lattice is distributive, while OCANNL's lattice is not distributive). Star does not aim at tracking array (dimension) sizes. Star does not have shape inference in the following sense: it does not synthesize new algebraic data types.

APL and J introduced the rank-polymorphic array-programming tradition: functions consume cells and are lifted over frames of extra axes, as formalized in Remora *Slepak, Shivers, Manolios, "The Semantics of Rank Polymorphism", 2019*. This is structural rank-polymorphism. A similarity with OCANNL is that the computation semantics are derived at function application site. In OCANNL, freshened constraints are introduced when a function computes tensor values (at OCaml runtime), these constraints are used both for shapes and for indexing (computation semantics). Refined Remora *Matviichuk, Shivers, "Refined Remora: Constraining Array Shapes", 2026* adds SMT-checked refinements over dimensions and shape sequences, bringing expressivity from below to beyond what's available in OCANNL. However, refinements are not inferred.

Vasilache, Zinenko, Theodoridis et al. "Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions", 2018 is the closest surface precedent: index variables induce ranges, right-hand-side-only indices induce reductions, affine subscripts express convolution, and underconstrained ranges require explicit annotations. Production systems propagate symbolic or partial shapes through graphs or IRs.

There is no rank-polymorphism. The notation is not as concise as in OCANNL: numeric precisions need to always be specified (in OCANNL they are inferred bidirectionally), and tensor fixed rank templates need to be declared.

Conclusions and future work

OCANNL uniquely combines parametric rank-polymorphism and structural rank-polymorphism, and offers powerful shape inference. It aims to cover most practical shape inference challenges, barring inherent incompleteness that would require backtracking.

The Appendix presents definitions and theorems showing termination and soundness, and illustrating the degree of (in)completeness. A full treatment with proofs is available at: ahrefs.github.io/ocannl/docs/pdfs/ocannl-formal-core-technical-report.pdf.

One problem to explore in OCANNL's shape inference is the current semantics of dimension basis comparisons: $1_{\emptyset} \neq 1_{\texttt{default}}$. For example, it makes scalars incompatible with dimension-1 data vectors as same-size arguments to einsum-based operations. This problem has not yet surfaced in practice: "it's a feature, not a bug". Once practical examples suffer from this incompatibility, they might motivate a more sophisticated approach (e.g. basis polymorphism).

OCANNL also intends to provide to the OCaml ecosystem a compilation path for tensor computations across various GPU backends: Nvidia (CUDA), Apple Silicon (Metal), AMD (HIP). The compiler performs inlining and Common Subexpression Elimination on the lowered IR (a loop nest language). Tiling for threadblocks and tensor cores, and further optimizations, are future work at the time of writing. We expect this effort to revolve around automated search over schedules and programs, in the line of tinygrad's BEAM search, *Ragan-Kelley et al., "Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines", PLDI 2013*, and *Baghdadi et al., "Tiramisu: A Polyhedral Compiler for Expressing Fast and Portable Code", CGO 2019*.

Authorship: the content above, was written by the human author without AI/LLM feedback, apart from a final AI-assisted review pass (Claude Fable 5) that caught typos and consistency issues against the technical report, and a draft of the Benchmarks section. The appendices below and the accompanying technical report were created by Claude Fable 5 (interactively, Appendix trimmed down for brevity) and GPT 5.5 (fixing issues I noticed, final compilation and proof gaps).

Appendix: Benchmark tables

This appendix continues from the Benchmarks section.

Every runner loads the same initial weights, data and hyperparameters from a shared safetensors fixture, and a parity gate compares the loss trajectory of the first steps against the PyTorch CPU reference — timing rows are comparable only because the math agrees (the parity column reports the maximum relative difference; REF marks the reference row). The gate doubled as a cross-framework correctness oracle: on its first run it caught two real backward-pass bugs in OCANNL's optimizer (both since fixed, and turned into a regression test).

Measurement is identical across frameworks: the device is synchronized around timed regions, warmup steps are untimed, and we report per-step wall-time percentiles (p50/p10/p90). One-time cost — graph build, code generation, JIT capture, or autotune search — is reported separately as compile seconds and never amortized into step time. OCANNL appears in three variants: *default* keeps intermediates virtual (recomputed in the backward pass), *materialized* materializes them, and *tuned* runs `Train.tune_placements`, an autotuning search over placement decisions that keeps the measured winner — the counterpart of tinygrad's BEAM search and `torch.compile`. Tuned step timings come from a fresh process replaying the cached winning schedule; the tuned row's compile seconds is the from-scratch search cost. Full reports (including enqueue-only timings and throughput) are checked into the repository.

Apple silicon, Metal backend

Measured on an Apple-silicon laptop (macOS 26.5, arm64); OCANNL backends `cc` (CPU) and `metal`, PyTorch devices `cpu` and `mps`, tinygrad devices `CPU` and `METAL`. Step times are milliseconds. The `compiled` rows (torch 2.13.0) and the `beam` rows (tinygrad BEAM=2) come from later single-framework reruns on the same machine, each parity-gated against its own CPU eager reference; the reruns' eager and jit timings reproduce the rows below within a few percent.

gpt2_mini (forward-only; the full report additionally lists tokens/s):

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
pytorch	mps	compiled	1.935	1.478	2.059	1.48	6.7e-08
tinygrad	METAL	beam	2.146	2.051	2.185	21.53	4.7e-07
tinygrad	METAL	jit	3.662	3.165	3.764	0.69	1.3e-07
pytorch	mps	eager	5.905	5.002	6.997	0.05	1.3e-07
pytorch	cpu	compiled	9.624	9.407	9.889	7.82	2.0e-07
pytorch	cpu	eager	13.457	13.323	13.671	0.02	REF
tinygrad	CPU	beam	27.124	27.042	27.189	80.14	8.0e-07
tinygrad	CPU	jit	45.157	44.748	45.377	0.79	8.0e-07
ocannl	metal	tuned	92.707	92.545	92.907	1947.18	2.0e-07
ocannl	metal	default	367.015	365.519	367.335	0.23	2.0e-07
ocannl	cc	tuned	381.585	379.912	382.755	150.71	8.1e-07
ocannl	metal	materialized	399.998	398.475	400.709	0.31	8.1e-07
ocannl	cc	default	2085.600	2084.080	2090.320	3.65	8.1e-07
ocannl	cc	materialized	2261.310	2258.720	2263.830	14.97	8.1e-07

lenet:

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
tinygrad	METAL	beam	0.980	0.520	1.058	57.16	2.1e-07
tinygrad	METAL	jit	1.176	0.764	1.313	1.30	2.1e-07
tinygrad	CPU	beam	2.882	2.796	2.986	205.16	3.1e-07
pytorch	mps	eager	5.639	5.495	5.830	0.10	1.0e-07
pytorch	mps	compiled	5.641	5.292	5.746	1.08	2.1e-07
tinygrad	CPU	jit	5.726	5.602	5.812	1.31	3.1e-07
pytorch	cpu	eager	12.527	12.245	12.775	0.02	REF
pytorch	cpu	compiled	12.803	12.538	13.060	5.58	1.0e-07
ocannl	cc	tuned	14.269	14.196	14.365	21.86	2.1e-07
ocannl	cc	materialized	14.301	14.207	14.373	1.65	2.1e-07
ocannl	cc	default	18.988	18.911	19.062	1.38	2.1e-07
ocannl	metal	tuned	36.612	36.316	37.129	72.69	2.1e-07
ocannl	metal	materialized	37.819	37.443	38.327	0.55	2.1e-07
ocannl	metal	default	216.387	215.266	217.781	0.55	2.1e-07

mlp_small:

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
ocannl	cc	tuned	0.129	0.107	0.148	7.70	2.2e-07
pytorch	cpu	compiled	0.156	0.132	0.195	3.16	3.2e-07

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
ocannl	cc	materialized	0.177	0.155	0.216	0.46	2.2e-07
pytorch	cpu	eager	0.178	0.161	0.221	0.01	REF
ocannl	cc	default	0.182	0.158	0.247	0.45	2.2e-07
tinygrad	CPU	beam	0.214	0.212	0.225	53.47	3.6e-07
tinygrad	CPU	jit	0.302	0.295	0.314	0.32	2.8e-07
tinygrad	METAL	beam	0.800	0.299	0.990	12.34	1.2e-07
tinygrad	METAL	jit	0.823	0.311	0.942	0.35	2.4e-07
pytorch	mps	compiled	1.048	0.938	1.120	0.57	2.4e-07
pytorch	mps	eager	1.134	0.990	1.235	0.07	1.2e-07
ocannl	metal	tuned	1.273	1.162	1.368	1.17	3.2e-07
ocannl	metal	materialized	1.300	1.213	1.383	0.01	3.2e-07
ocannl	metal	default	1.508	1.326	1.893	0.01	3.2e-07

mlp_wide:

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
pytorch	mps	compiled	1.048	0.982	1.127	0.59	2.1e-07
pytorch	mps	eager	1.079	0.941	1.131	0.08	2.1e-07
tinygrad	METAL	beam	1.095	0.615	1.212	18.65	3.1e-07
tinygrad	METAL	jit	1.168	0.744	1.290	0.52	2.1e-07
pytorch	cpu	eager	1.608	1.582	1.634	0.01	REF
pytorch	cpu	compiled	1.750	1.717	1.788	3.10	2.1e-07
ocannl	metal	tuned	4.144	4.091	4.313	510.25	5.2e-07
ocannl	metal	materialized	5.946	5.850	6.287	0.02	5.2e-07
ocannl	metal	default	6.281	6.168	6.591	0.02	5.2e-07
tinygrad	CPU	beam	6.702	6.377	6.916	66.08	7.3e-07
tinygrad	CPU	jit	13.441	13.220	13.842	0.49	7.3e-07
ocannl	cc	tuned	49.034	48.413	49.787	14.55	5.2e-07
ocannl	cc	materialized	218.940	218.124	219.787	0.57	6.2e-07
ocannl	cc	default	219.023	218.179	220.031	0.54	6.2e-07

NVIDIA, CUDA backend

Measured on an NVIDIA GeForce RTX 3050 Ti Laptop GPU (4 GB) under WSL2 (Linux x86_64); torch 2.13.0+cu130, tinygrad 0.13.0, CUDA 12.8. Step times are milliseconds. The `compiled` rows (torch 2.13.0+cu130) and the `beam` rows (tinygrad BEAM=2, tinygrad 0.13.0) come from later single-framework reruns on the same machine at the same framework versions, each parity-gated against its own CPU eager reference. This thermally limited machine shows sizable run-to-run variance on CPU-bound cells (up to roughly 1.7x between the original run and the reruns; GPU cells reproduce within about 10%), so comparisons across rows are indicative. In several cells the BEAM-searched kernels time slower than plain JIT, consistent with throttling right after the long search phase; the lenet speedups survive it.

gpt2_mini (forward-only; the full report additionally lists tokens/s):

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
pytorch	cuda	compiled	3.507	3.319	3.567	5.67	6.7e-08
pytorch	cuda	eager	8.172	7.615	8.689	0.23	6.7e-08

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
tinygrad	CUDA	jit	12.830	11.786	12.922	1.03	1.3e-07
tinygrad	CUDA	beam	15.844	15.769	16.065	62.87	5.4e-07
pytorch	cpu	compiled	42.936	35.991	45.764	9.05	1.3e-07
pytorch	cpu	eager	82.427	80.647	86.397	0.10	REF
tinygrad	CPU	beam	143.347	132.362	150.304	125.59	8.7e-07
tinygrad	CPU	jit	157.738	153.594	159.969	1.41	8.7e-07
ocannl	cuda	default	274.629	274.563	274.788	1.40	8.7e-07
ocannl	cuda	tuned	275.133	275.002	275.299	633.80	8.7e-07
ocannl	cuda	materialized	330.590	330.364	331.165	2.67	8.7e-07
ocannl	cc	tuned	1212.400	1113.860	1256.160	148.66	9.4e-07
ocannl	cc	default	2330.670	2275.380	2355.680	2.28	8.7e-07
ocannl	cc	materialized	2590.840	2568.950	2605.810	7.27	9.4e-07

lenet:

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
tinygrad	CUDA	beam	0.736	0.703	4.060	200.54	3.1e-07
tinygrad	CUDA	jit	1.296	1.085	1.317	2.18	2.1e-07
pytorch	cuda	eager	1.782	1.669	2.701	0.56	8.8e-05
pytorch	cuda	compiled	2.174	1.576	2.359	6.16	8.8e-05
pytorch	cpu	eager	3.652	3.247	4.005	0.18	REF
pytorch	cpu	compiled	4.349	4.116	4.627	5.17	2.1e-07
tinygrad	CPU	beam	5.226	5.055	5.381	340.88	3.1e-07
ocannl	cuda	tuned	18.514	18.456	18.567	89.13	2.1e-07
ocannl	cuda	materialized	18.580	18.539	18.635	1.44	2.1e-07
tinygrad	CPU	jit	22.030	21.481	22.639	2.17	3.1e-07
ocannl	cc	tuned	23.509	22.934	24.820	48.58	3.1e-07
ocannl	cc	materialized	23.876	23.324	25.042	3.29	3.1e-07
ocannl	cc	default	40.553	39.790	42.219	2.49	3.1e-07
ocannl	cuda	default	174.170	174.095	174.252	1.26	2.1e-07

mlp_small:

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
ocannl	cc	tuned	0.151	0.147	0.164	10.36	2.8e-07
ocannl	cc	materialized	0.166	0.161	0.175	0.58	2.8e-07
ocannl	cuda	tuned	0.169	0.122	0.323	34.52	2.2e-07
tinygrad	CUDA	jit	0.179	0.175	0.217	0.50	2.8e-07
ocannl	cuda	default	0.180	0.126	0.303	0.13	2.2e-07
ocannl	cc	default	0.181	0.176	0.193	0.56	2.8e-07
ocannl	cuda	materialized	0.203	0.152	0.305	0.14	2.2e-07
pytorch	cpu	eager	0.253	0.231	0.636	0.15	REF
tinygrad	CPU	beam	0.348	0.340	0.402	89.53	2.4e-07
tinygrad	CUDA	beam	0.354	0.347	0.429	39.47	2.4e-07
pytorch	cpu	compiled	0.411	0.370	0.470	3.27	2.2e-07
tinygrad	CPU	jit	0.680	0.638	0.749	0.56	2.4e-07
pytorch	cuda	compiled	1.088	0.940	1.369	2.16	1.2e-07

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
pytorch	cuda	eager	1.296	0.981	1.573	0.17	1.2e-07

mlp_wide:

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
pytorch	cuda	eager	1.110	1.067	1.415	0.17	1.0e-07
pytorch	cuda	compiled	1.415	1.341	1.526	3.64	2.1e-07
tinygrad	CUDA	jit	2.749	2.732	3.314	0.67	1.0e-07
tinygrad	CUDA	beam	2.904	2.862	3.051	76.87	4.1e-07
pytorch	cpu	compiled	11.485	9.872	12.268	3.14	1.0e-07
pytorch	cpu	eager	11.554	10.863	12.394	0.13	REF
ocannl	cuda	tuned	13.912	13.604	14.212	453.11	5.1e-07
ocannl	cuda	default	14.120	13.855	14.373	0.21	5.1e-07
ocannl	cuda	materialized	14.215	13.982	14.477	0.18	5.1e-07
tinygrad	CPU	beam	27.802	27.101	29.370	144.05	6.2e-07
tinygrad	CPU	jit	43.799	42.511	45.605	0.86	6.2e-07
ocannl	cc	tuned	89.214	85.214	116.715	21.66	5.2e-07
ocannl	cc	default	286.176	275.921	322.762	0.73	5.2e-07
ocannl	cc	materialized	289.759	278.446	322.250	0.73	5.2e-07

AMD, HIP backend

Measured on an AMD Strix Halo (Radeon 8060S iGPU, gfx1151) under Windows 11, ROCm/HIP SDK 7.1; OCANNL backends `cc` (CPU) and `hip`. Coverage is partial: on Windows neither PyTorch nor tinygrad reaches the AMD GPU, so PyTorch appears only as the CPU eager parity reference, and the compiled/BEAM variants are absent. Step times are milliseconds.

gpt2_mini (forward-only; the full report additionally lists tokens/s):

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
pytorch	cpu	eager	23.356	22.367	24.945	0.03	REF
ocannl	hip	tuned	67.331	66.451	70.356	542.34	1.3e-07
ocannl	hip	default	68.278	66.471	71.358	1.96	1.3e-07
ocannl	hip	materialized	72.636	71.460	73.412	2.73	8.7e-07
ocannl	cc	tuned	500.138	486.113	509.318	151.19	8.0e-07
ocannl	cc	default	2212.050	2209.280	2214.540	3.14	8.7e-07
ocannl	cc	materialized	2387.700	2385.920	2389.150	10.01	8.0e-07

lenet:

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
pytorch	cpu	eager	1.371	1.246	1.755	0.01	REF
ocannl	hip	materialized	22.246	21.962	22.622	1.75	3.1e-07
ocannl	hip	tuned	22.318	21.994	22.616	89.72	3.1e-07
ocannl	cc	materialized	22.509	22.374	22.752	3.35	3.1e-07

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
ocannl	cc	tuned	22.644	22.477	23.071	196.90	3.1e-07
ocannl	cc	default	32.370	32.121	32.684	3.24	3.1e-07
ocannl	hip	default	82.096	81.653	82.477	1.50	2.1e-07

mlp_small:

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
ocannl	cc	tuned	0.107	0.106	0.115	16.66	2.8e-07
ocannl	cc	materialized	0.139	0.137	0.142	0.92	2.8e-07
ocannl	cc	default	0.140	0.135	0.148	0.82	2.8e-07
ocannl	hip	default	0.190	0.182	0.213	0.09	3.0e-07
ocannl	hip	materialized	0.208	0.199	0.241	0.09	2.8e-07
ocannl	hip	tuned	0.210	0.199	0.257	5.65	3.0e-07
pytorch	cpu	eager	0.268	0.239	0.310	0.00	REF

mlp_wide:

framework	backend	variant	step p50 ms	p10	p90	compile s	parity
ocannl	hip	tuned	1.630	1.584	1.730	114.01	4.2e-07
ocannl	hip	materialized	1.750	1.703	1.823	0.40	4.2e-07
ocannl	hip	default	1.832	1.766	2.038	0.40	4.2e-07
pytorch	cpu	eager	3.984	3.587	4.720	0.01	REF
ocannl	cc	tuned	130.773	129.064	134.170	27.34	4.2e-07
ocannl	cc	materialized	331.161	329.459	333.288	1.21	5.2e-07
ocannl	cc	default	331.693	329.856	333.346	1.18	5.2e-07

Appendix: Shape and Projections inference: semantics and correctness

TR refers to ahrefs.github.io/ocannl/docs/pdfs/ocannl-formal-core-technical-report.pdf. Notation (shared with the TR): $\sqsubseteq$ is the algebraic order, on dimensions and on marked rows; $\preceq$ is the row subtype/refinement relation of TR Definition 2.12, the relation used when generating row constraints — the algebraic order on rows is too strong for the broadcast semantics; $\leq$ is reserved for the substitution preorder.

1. Dimensions

Definition [TR Definition 1.1] (Dimensions). Fix a set $\mathcal{B}$ of basis tags with a distinguished tag `default` $\in \mathcal{B}$. The set of dimensions is

$$D \;=\; \{1_\emptyset\} \;\cup\; \{\, n_b \mid n \geq 1,\; b \in \mathcal{B} \,\}.$$

Elements n_b are *atoms*; $1_\emptyset$ is the *claim-free unit*. Adjoin a fresh element $\bot$ and set $D_\bot = D \cup \{\bot\}$.

Definition [TR Definition 1.2] (Broadcast order). On D: $d_1 \sqsubseteq d_2$ iff $d_2 = 1_\emptyset$ or $d_1 = d_2$. Extend to $D_\bot$ by $\bot \sqsubseteq d$ for all d.

Proposition [TR Proposition 1.3] (Lattice structure). $(D_\perp, \sqsubseteq)$ is a bounded lattice of height 2: top $1_\emptyset$, bottom $\perp$, and the atoms form an antichain, each covering $\perp$ and covered by $1_\emptyset$. Meets and joins:

$$d \wedge d = d, \quad d \wedge 1_\emptyset = d, \quad d_1 \wedge d_2 = \perp \ (d_1 \neq d_2 \text{ atoms}); \qquad d \vee d = d, \quad d \vee \perp = d, \quad d_1 \vee d_2 = 1_\emptyset \ (d_1 \neq d_2 \text{ atoms}).$$

Proposition [TR Proposition 1.4] (Non-distributivity). D contains at least three atoms, e.g. $\{1_{\texttt{default}}, 2_{\texttt{default}}, 3_{\texttt{default}}\} \subseteq D$, so $D_\perp$ is not distributive (it is, however, modular).

2. Rows

Definition [TR Definition 2.1] (Rows). A *row* is a triple $l \cdot \diamond \cdot r$ with $l, r \in D^*$ (the *leading* and *trailing flanks*). $\mathrm{rank}(l \cdot \diamond \cdot r) = |l| + |r|$. Equality of rows is componentwise equality of (l, r) — in particular **marker-sensitive**: $[3] \cdot \diamond \cdot [4] \neq [3, 4] \cdot \diamond \cdot [\,]$. (This is identity of the algebra's elements; it is *not* the satisfaction relation of equality *constraints*, which TR Definition 3.4 judges by the flat equivalence $\approx$ of TR Definition 2.8.)

Definition [TR Definition 2.2] (Expansion). For a row $R = l \cdot \diamond \cdot r$ and $n \geq \mathrm{rank}(R)$, the expansion $R{\uparrow}n \in D^n$ is the flat sequence $l \cdot (1_\emptyset)^{\,n - \mathrm{rank}(R)} \cdot r$.

Definition [TR Definition 2.3] (Row order). $l_2 \cdot \diamond \cdot r_2 \sqsubseteq l_1 \cdot \diamond \cdot r_1$ iff (i) $|l_1| \leq |l_2|$ and $|r_1| \leq |r_2|$ (*flanks fit*; note this implies $\mathrm{rank}_2 \geq \mathrm{rank}_1$), and (ii) $(l_2 \cdot r_2)[i] \sqsubseteq (l_1 \cdot \diamond \cdot r_1){\uparrow}\mathrm{rank}_2\,[i]$ for all i (*pointwise refinement of the expansion*).

Lemma [TR Lemma 2.4] (Expansion monotonicity). If $R_2 \sqsubseteq R_1$ then for every $n \geq \mathrm{rank}(R_2)$: $R_2{\uparrow}n\,[i] \sqsubseteq R_1{\uparrow}n\,[i]$ for all i.

Proposition [TR Proposition 2.5] (Partial order). $\sqsubseteq$ is a partial order on rows.

Proposition [TR Proposition 2.6] (The empty row is the greatest element). For every row R: $R \sqsubseteq [\,] \cdot \diamond \cdot [\,]$, and nothing else is above all rows.

Proposition [TR Proposition 2.7] (Rows form a join-semilattice with top). Any two rows R_1, R_2 have a least upper bound:

$$R_1 \vee R_2 \;=\; l_\vee \cdot \diamond \cdot r_\vee, \qquad |l_\vee| = \min(|l_1|, |l_2|), \quad l_\vee[i] = l_1[i] \vee l_2[i],$$

and symmetrically for $r_\vee$ aligned from the back (joins taken in D; note two distinct atoms join to $1_\emptyset$, so $l_\vee \in D^*$).

Remark [TR §2] (Rank-polymorphism lives in the order). TR Definition 2.3 relates rows of different ranks directly; no quantifier appears anywhere in this section.

Definition [TR Definition 2.8] (Flat equivalence). $R \approx R'$ iff $\mathrm{flat}(R) = \mathrm{flat}(R')$ as sequences, where $\mathrm{flat}(l \cdot \diamond \cdot r) = l \cdot r$ — equivalently, $\mathrm{rank}(R) = \mathrm{rank}(R')$ and $R{\uparrow}\mathrm{rank}(R) = R'{\uparrow}\mathrm{rank}(R')$, since a row's expansion at its own rank is its flattening (TR Definition 2.2 with no padding). $\approx$ is the marker-erasing equivalence; it is what the implementation's closed-row "equality" computes (§10).

Proposition [TR Proposition 2.9] ($\approx$ versus the order). (i) The equivalence induced by $\sqsubseteq$ (mutual inequality) is the **identity** — this is just antisymmetry (TR Proposition 2.5). So $\approx$ is not derivable from the order. (ii) Stronger: any two *distinct* $\approx$-related rows are $\sqsubseteq$-**incomparable**. (iii) $\approx$ is not a $\sqsubseteq$-congruence; indeed no nontrivial equivalence is compatible with the order. (iv) $\approx$ is the kernel of expansion at own rank; everything else the order consults — flank-fit, and expansions at strictly higher ranks (broadcasting) — is exactly where $\approx$-related rows diverge.

The implementation does not use the observational equivalence of rows-as-operands: its equality sites use $\approx$ (observational equivalence is finer), its row-subtyping sites use the marker-erasing-lower relation of TR Definition 2.12 (observational equivalence is looser because it absorbs explicit inner-edge $1_\emptyset$ entries into the middle).

Remark [TR after Proposition 2.9; §6.1] (Equality-as-$\approx$ is underdetermined; the marker placement is a policy choice). Reading the ground meaning of closed-row equality as $\approx$ (forced by

practice: an einsum spec row must equate with a literal shape whose marker sits at the front — §10) makes row-equality constraints underdetermined: in $l_1 \cdot \langle\rho\rangle \cdot r_1 \approx C$, the flat content of $\gamma\rho$ is forced (the middle of flat(C)) but its marker placement is free, and by TR Proposition 2.9(ii) the candidates are pairwise $\sqsubseteq$-incomparable in the internal marked order — a later row-subtyping constraint with ρ on the upper/operand side can distinguish them. Witness: $\Phi = \{\, \langle\rho\rangle \approx [\,] \cdot \diamond \cdot [3,5],\ \ [3] \cdot \diamond \cdot [9,5] \preceq \langle\rho\rangle \,\}$. Under $\approx$, $\gamma\rho = [3] \cdot \diamond \cdot [5]$ satisfies both ($[3,9,5]$ vs $[3,1_\emptyset,5]$). The implementation is $\approx$*-checking but marked-committing*: it accepts the flat alignment, then commits the closed side's structural split as the value's marker placement — on this Φ it fails. TR Definition 3.4 now adopts the $\approx$ reading and TR Definition 2.12's $\preceq$ reading for row-subtyping constraints, so that failure is officially a *policy rejection* (TR Lemma 5.1's policy steps), not semantic unsatisfiability: closed-row equality joins closing (the non-principality remark below) as a deliberately non-principal site.

Definition [TR Definition 2.10] (Two-sorted ground rows). Adjoin to the marked rows of TR Definition 2.1 a second sort of *rigid* rows: $F^\bullet$ with $F \in D^n$, a flat sequence with **no marker**; $\mathrm{rank}(F^\bullet) = n$, and the expansion $F^\bullet{\uparrow}m$ is defined only at $m = n$, where it is F. Extend $\sqsubseteq$: (a) marked–marked: TR Definition 2.3 unchanged; (b) $F_2^\bullet \sqsubseteq F_1^\bullet$ iff equal rank and pointwise refinement; (c) $F^\bullet \sqsubseteq R$ (R marked) iff $\mathrm{rank}(F) \geq \mathrm{rank}(R)$ and $F[i] \sqsubseteq R{\uparrow}\mathrm{rank}(F)\,[i]$ pointwise — a rigid result may refine a broadcastable operand; (d) $R \sqsubseteq F^\bullet$: **never** — nothing broadcastable sits below a rigid row.

Proposition [TR Proposition 2.11] (The two-sorted order). The extended relation is a partial order; the empty marked row remains the unique top; and on the rigid sort, flat equality is trivially a congruence, since no context consults a marker that does not exist. Admitting even rank-equal $R \sqsubseteq F^\bullet$ would break transitivity: $[5] \cdot \diamond \cdot [3] \sqsubseteq [\,] \cdot \diamond \cdot [3] \sqsubseteq [3]^\bullet$ would demand $[5] \cdot \diamond \cdot [3] \sqsubseteq [3]^\bullet$, a rank mismatch against a rigid row. The desired surface relation exists in the system as the derived row subtype/refinement relation $R \preceq S$ of TR Definition 2.12, defined by $\mathrm{flat}(R)^\bullet \sqsubseteq S$.

3. Terms, substitutions, and constraint derivation

Definition [TR Definition 3.1] (Terms). Dimension terms $t ::= \alpha \mid n_b \mid 1_\emptyset$. Row terms $R ::= l \cdot \diamond \cdot r \mid l \cdot \langle\rho\rangle \cdot r$ with l, r sequences of dimension terms; we identify a bare row variable ρ with $[\,] \cdot \langle\rho\rangle \cdot [\,]$. A term is *ground* if variable-free. Each row term contains **at most one** row variable, at the marker — an invariant preserved by everything below.

Definition [TR Definition 3.2] (Substitution). A substitution σ is a finite, sort-respecting map from variables to terms with $\mathrm{dom}(\sigma) \cap \mathrm{vars}(\mathrm{ran}(\sigma)) = \emptyset$ (idempotency). Application is structural except at the marker: if $\sigma(\rho) = l' \cdot \langle\rho'\rangle \cdot r'$ (or closed, $l' \cdot \diamond \cdot r'$), then

$$\sigma(l \cdot \langle\rho\rangle \cdot r) \;=\; (\sigma l \cdot l') \cdot \langle\rho'\rangle \cdot (r' \cdot \sigma r) \quad \big(\text{resp. } (\sigma l \cdot l') \cdot \diamond \cdot (r' \cdot \sigma r)\big).$$

Lemma [TR Lemma 3.3] (Composition). $(\sigma \circ \tau)(X) = \sigma(\tau(X))$ for all terms X, where $\sigma \circ \tau$ is the usual composite substitution.

Definition [TR Definitions 3.4-3.5] (Semantics). A *ground substitution* γ is total on a fixed countable variable universe and maps into ground dimensions/rows. For a constraint set Φ over atomic constraints $t_1 = t_2 \mid t_1 \sqsubseteq t_2 \mid R_1 \approx R_2 \mid R_1 \preceq R_2$:

$$\mathrm{Sol}(\Phi) = \{\gamma \text{ ground} \mid \gamma(\phi) \text{ holds for every } \phi \in \Phi\},$$

where ground *dimension* equalities are identity, ground *row* equalities are judged by the flat equivalence $\approx$ of TR Definition 2.8, equivalently, identity of the rigidifications $\mathrm{flat}(\cdot)^\bullet$ in the two-sorted algebra of TR Definition 2.10, equivalently mutual $\preceq$; the three formulations coincide. Ground row-subtyping constraints use the row subtype/refinement relation $\preceq$ of TR Definition 2.12. A substitution σ *models* Φ, $\sigma \models \Phi$, iff $\gamma \circ \sigma \in \mathrm{Sol}(\Phi)$ for every ground γ, equivalently, $\sigma(\Phi)$ is valid under universal quantification of its free variables. The *substitution order*: $\sigma_1 \leq \sigma_2$ iff $\exists u.\ u \circ \sigma_1 = \sigma_2$.

Example [TR Example 3.6] (No principal model exists in general). Let $\Phi = \{3_b \sqsubseteq \alpha\}$. The identity does not model Φ (ground $\alpha \mapsto 5_b$ falsifies it). Any model must map α to a term all of whose groundings

lie above 3_b, i.e. to 3_b or $1_\emptyset$ (a variable target fails as the identity did). The two models $\sigma_1 = [\alpha \mapsto 3_b]$ and $\sigma_2 = [\alpha \mapsto 1_\emptyset]$ are $\leq$-incomparable: a witness u for either direction would have to send a ground dimension to a different ground dimension, which substitutions cannot do. Hence Φ has models but **no $\leq$-least model**.

Consequence. The solver's answer is a pair — a substitution *and a residual bound store* — and the correct theorem is about solution-set representation and most-generality (TR Theorem 5.9 below).

Compact rule table (constraint derivation). Let C be the result shape and A, B, D operand shapes, with S_k denoting the row of kind $k \in \{B, I, O\}$ (batch, input, output). For spec-based rules, let T_k^{lhs} and $T_{i,k}^{\text{rhs}}$ be the rows obtained by elaborating the result and operand slots of the specification. The generated core constraints are:

Operation logic	Generated constraints
transpose	$C_B \preceq A_B, \quad C_I \preceq A_O, \quad C_O \preceq A_I$
pointwise unary	$C_k \preceq A_k$ for every k
pointwise binary	$C_k \preceq A_k, \quad C_k \preceq B_k$ for every k
pointwise ternary	$C_k \preceq A_k, \quad C_k \preceq B_k, \quad C_k \preceq D_k$ for every k
compose	$A_I \preceq B_O, \quad C_B \preceq A_B, \quad C_B \preceq B_B, \quad C_I \preceq B_I, \quad C_O \preceq A_O$
compose accumulate	the compose constraints, plus $C_k \preceq D_k$ for every k
batch slice	$(s \cdot C_B) \approx A_B, \quad C_I \approx A_I, \quad C_O \approx A_O$, where s is the sliced outer batch axis
permute	$C_k \approx T_k^{\text{lhs}}, \quad T_{1,k}^{\text{rhs}} \approx A_k$ for every k
einsum	$C_k \approx T_k^{\text{lhs}}, \quad T_{i,k}^{\text{rhs}} \approx A_{i,k}$ for every operand i and kind k

Here $\preceq$ is the row subtype/refinement relation of TR Definition 2.12, while $\approx$ is flat row equivalence. Thus pointwise, transpose, and compose operations generate the broadcast-checking constraints of the core; permute and einsum deliberately generate equalities, the stronger inference policy used to recover labels and row variables bidirectionally.

4. The solver: configurations and rules

Definition [TR Definition 4.1] (Configuration). A configuration is $\langle \Phi;\ \sigma;\ B \rangle$: unsolved constraints, accumulated (idempotent) bindings, and a *bound store*. B assigns to each unsolved variable: for a dimension variable α, at most one atom lower bound $\text{lb}(\alpha) \in \{n_b\}$ plus a set of deferred variable–variable inequalities ("adjacencies"); for a row variable ρ, a set of row-term lower bounds ("caps", from inequality residues) plus adjacencies. There is a distinguished failure configuration fail.

Binding a variable ($\sigma := [x \mapsto T] \circ \sigma$) **re-emits** into Φ every bound and adjacency stored for x, instantiated at T, and substitutes T for x throughout Φ and B. This single discipline replaces ad-hoc propagation.

At dimension sort, the stored $\text{glb}(\alpha)$ is understood as the join of all ground lower bounds known for α, including those entailed through directed adjacencies. Operationally, when $d \sqsubseteq \alpha$ is recorded, the solver enqueues $d \sqsubseteq \beta$ for every stored edge $\alpha \sqsubseteq \beta$; when a new edge $\alpha \sqsubseteq \beta$ is recorded, it enqueues every stored cap $d \sqsubseteq \alpha$ as $d \sqsubseteq \beta$. Fair processing iterates these emissions to fixpoint. This is not a closing policy: it is the transitivity step $d \sqsubseteq \alpha \sqsubseteq \beta \Rightarrow d \sqsubseteq \beta$, and DI-cap handles any collapse produced downstream.

Definition [TR §4.1; TR Lemma 4.2] (Dimension rules).

	Constraint	Action
DE-refl	$t = t$	discard
DE-clash	$n_b = m_c$, $(n, b) \neq (m, c)$ (incl. atom vs $1_\emptyset$)	fail
DE-bind	$\alpha = t$, $t \neq \alpha$	bind $\alpha \mapsto t$
DI-top	$t \sqsubseteq 1_\emptyset$	discard
DI-refl	$t \sqsubseteq t$	discard

	Constraint	Action
DI-ground	$d \sqsubseteq d'$ ground	check TR Definition 1.2; else fail
DI-pin	$\alpha \sqsubseteq d$, d an atom	replace by $\alpha = d$
DI-pin-top	$1_\emptyset \sqsubseteq \alpha$	replace by $\alpha = 1_\emptyset$
DI-cap	$d \sqsubseteq \alpha$, d an atom	if $\mathrm{lb}(\alpha)$ unset: record and forward along stored outgoing adjacencies; if $= d$: discard; if $= d' \neq d$: replace by $\alpha = 1_\emptyset$
DI-adj	$\alpha \sqsubseteq \beta$, $\alpha \neq \beta$	defer (record adjacency on both) and forward α's stored cap, if any, to β

Justifications (each rule preserves Sol*):* DI-pin: in D, the only element $\sqsubseteq$ an atom d is d itself. DI-pin-top: the only element above the top is the top. DI-cap collapse: any $\gamma(\alpha)$ above two distinct atoms must be $1_\emptyset$ (TR Proposition 1.3). DI-cap/DI-adj forwarding emits only transitive consequences of already-stored constraints.

Definition [TR Definitions 4.3-4.4; TR Lemma 4.5] (Row rules). Write both sides aligned at the **outer edges**: leading flanks from the front, trailing flanks from the back.

For equality, first emit dimension equalities on the overlapping aligned flank positions. Let m_l, m_r be the surpluses: the unmatched inner segments of the longer leading and trailing flanks. "Defer into the closing policy" means postpone a constraint till a later stage of the solving process.

Equality $R_1 \approx R_2$.

	Situation	Action
RE-closed	both sides closed	fail iff the ranks differ; otherwise zip the flattened rows, emitting $\mathrm{flat}(R_1)[i] = \mathrm{flat}(R_2)[i]$
RE-open-closed	one side open with ρ, the other closed	fail iff the open side's total explicit flank length exceeds the closed side's rank; otherwise bind ρ to the uncovered middle of $\mathrm{flat}(R_{\mathrm{closed}})$, placed by the inherit-the-split policy
RE-nested	both sides open, $\rho_1 \neq \rho_2$, and both surpluses are on ρ_2's side	bind $\rho_1 \mapsto m_l \cdot \langle \rho_2 \rangle \cdot m_r$
RE-cross	both sides open, $\rho_1 \neq \rho_2$, with split surpluses	defer the residual word equation $s \cdot x_1 = x_2 \cdot t$ into the closing policy
RE-same-empty	both sides open with the same ρ and no surplus	discard
RE-same-occurs	both sides open with the same ρ and unequal known flank totals	fail
RE-same-rot	both sides open with the same ρ and equal totals but shifted splits	defer the rotational equation $x \cdot t = s \cdot x$ into the closing policy

Justifications and policy choices. RE-closed is exactly TR Definition 3.4's $\approx$-semantics: markers are not consulted. In RE-open-closed, substitution extends the open side's flanks inward only, so no grounding can shorten them. The binding $\rho \mapsto m_l \cdot \diamond \cdot m_r$ with $m_l \cdot m_r = m$ additionally chooses a placement that $\approx$ leaves free (the marker-placement remark above): it commits to the closed side's declared split where that split falls inside m, and to the left edge otherwise. This is a policy step in the family of TR Definition 6.1.

RE-nested is entailed by cancelling common outer flanks of the flat words; only placement is policy, as above. RE-cross is the two-variable word equation case: its solutions are the principal family $x_2 = s \cdot w$, $x_1 = w \cdot t$ **plus** at most $|s|$ *sporadic* cross-overlap solutions (x_2 a proper prefix of s, the rest forced). Consider an alternative: fresh-variable binding $\rho_2 \mapsto s \cdot \langle \rho' \rangle$, $\rho_1 \mapsto \langle \rho' \rangle \cdot t$. It captures exactly the principal family: exact under the former marked semantics, but under $\approx$ it forecloses the sporadic solutions, a *rank* commitment stronger than a placement choice (and it breaks rank conservativity of TR Lemma 5.3). Deferring keeps the equation in flight; substituting either variable makes the check exact, and the closing stage's upward close selects the corresponding extremal solution (for the variable carrying x_2, the outermost *sporadic* one), then the re-emitted equation validates it.

RE-same-occurs fails because no finite row satisfies $\rho \approx m_l \cdot \langle \rho \rangle \cdot m_r$ with $m_l \cdot m_r$ nonempty. RE-same-rot is different. Its residue $x \cdot t = s \cdot x$ is satisfiable under TR Definition 3.4's $\approx$-semantics exactly for conjugate s, t with cyclically periodic x (Lyndon–Schützenberger), a solution family outside the solver's constraint language. Deferring lets other constraints solve ρ and make the substituted closed-closed flat check exact. Otherwise, guessing closes ρ upward — the least-material disjunct, $x = [\,]$ — after which the equation requires $s = t$. This accepts exactly the $\delta = 0$ conjugacy class: a policy commitment in the family of TR Definition 6.1 and since $\approx$ is the adopted semantics, a genuine incompleteness on nontrivial conjugate instances (periodic $x \neq [\,]$), accepted deliberately.

For inequality, first emit dimension inequalities $t_{\text{res}} \sqsubseteq t_{\text{op}}$ on aligned flank overlaps.

Row subtype/refinement $R_{\text{res}} \preceq R_{\text{op}}$.

	Situation	Action
RI-cap	operand open, and result supplies at least the operand's flanks	record the result's interior residue as a **cap** on ρ_{op}; the residue is a row term, open if the result is open
RI-closed-op	operand closed, and result rank is at least the operand flanks	discard the operand's interior expansion; it is $1_\emptyset$, and the corresponding result positions are unconstrained
RI-short-closed	result closed and shorter than the operand's known flanks	fail
RI-deficit	result open and shorter than the operand's known flanks, with deficit $k > 0$	bind $\rho_{\text{res}} \mapsto \alpha_1 \cdots \alpha_{k_l} \cdot \langle \rho' \rangle \cdot \beta_1 \cdots \beta_{k_r}$ with fresh dimension variables and a fresh row variable, sized to clear the deficit; then reprocess

RI-short-closed is a genuine rank mismatch. RI-deficit is subject to the rank-cycle check of TR Definition 4.6.

Definition [TR §5] (Fairness). A run processes constraints until $\Phi = \emptyset$ or fail; re-emitted constraints return to Φ. Any fair strategy is allowed.

Example [TR §4.3] (Divergence without a cycle check). Let $\Phi_0 = \{\, \rho_1 \preceq [a] \cdot \langle \rho_2 \rangle,\ \ \rho_2 \preceq [b] \cdot \langle \rho_1 \rangle \,\}$ (a, b ground atoms). Semantically: the first constraint forces $\text{rank}(\gamma\rho_1) \geq 1 + \text{rank}(\gamma\rho_2)$ by TR Definition 2.12, the second forces $\text{rank}(\gamma\rho_2) \geq 1 + \text{rank}(\gamma\rho_1)$; hence $\text{Sol}(\Phi_0) = \emptyset$. Operationally, RI-deficit fires on ρ_1, growing it by one axis; substitution lengthens the *operand* side of the second constraint, where RI-deficit grows ρ_2; substitution lengthens the operand side of the first constraint's residue, and so on forever. The dimension and row rules alone do not terminate. The self-cyclic instance $\rho \preceq [a] \cdot \langle \rho \rangle$ (obtainable by an einsum equality merging two row variables) diverges the same way. Developing this formalization uncovered an implementation bug: due to other mechanisms, the minimal *live* divergence was the three-variable cycle $\{\rho_1 \preceq \langle \rho_2 \rangle \cdot [a],\ \rho_2 \preceq \langle \rho_3 \rangle \cdot [b],\ \rho_3 \preceq \langle \rho_1 \rangle \cdot [c]\}$, which ran unboundedly until killed (now rejected by the TR Definition 4.6 check).

Definition [TR Definition 4.6] (Rank-fact graph and cycle check). Maintain beside the configuration a directed graph G on row variables — **persistent**: edges are only ever added, never retracted, in particular not when a variable is solved and substituted away. An edge $\rho \xrightarrow{k} \rho'$ with weight $k \geq 0$ records the entailed fact $\mathrm{rank}(\gamma\rho) \geq \mathrm{rank}(\gamma\rho') + k$ (for every solution γ of the current configuration). Edges are recorded at three points:

(i) *binding:* solving $\rho \mapsto l \cdot \langle \rho' \rangle \cdot r$ records $\rho \xrightarrow{|l|+|r|} \rho'$ (the entailed fact is an equality; one direction suffices for the check; this covers RE-nested bindings and open row bindings produced by RI-deficit);
(ii) *equal flanks:* processing $R_{\mathrm{res}} \preceq R_{\mathrm{op}}$ with both sides open and equal known flank lengths records $\rho_{\mathrm{res}} \xrightarrow{0} \rho_{\mathrm{op}}$;
(iii) *deficit:* RI-deficit with deficit $k > 0$ and open operand records $\rho_{\mathrm{res}} \xrightarrow{k} \rho_{\mathrm{op}}$ *before* growing (a closed operand needs no edge: the rank bound is absolute and the growth is one-shot).

Every insertion is **guarded**: if the new edge closes a directed cycle of total weight > 0, then fail.

Soundness of failing: summing the facts around a positive cycle gives $\mathrm{rank}(\gamma\rho) \geq \mathrm{rank}(\gamma\rho) + w$ with $w > 0$, so $\mathrm{Sol} = \emptyset$. *Soundness of persistence:* each edge is entailed by constraints present when it was recorded, and the solver never retracts a constraint — TR Lemma 5.1 only moves and re-expresses them — so entailed facts stay entailed, even when the variables involved have long been substituted away. Zero-weight cycles are legal: they assert rank *equality* along the cycle (and could be collapsed to row equalities, generalizing the one-step row antisymmetry the implementation already performs — cf. §10).

Sign discipline. In RI-cap — operand open, result's known flanks in *surplus* by $s > 0$ — the constraint entails only $\mathrm{rank}(\gamma\rho_{\mathrm{res}}) \geq \mathrm{rank}(\gamma\rho_{\mathrm{op}}) - s$: a *nonpositive* lower bound. Recording it with weight 0 (or s) is **unsound**: it manufactures cycles that no solution violates. A first implementation attempt did exactly this and rejected legitimate programs (the SGD self-reference idiom among them); the fixed implementation records nothing there. Recording a genuinely negative weight would be sound but is unnecessary for the check's purpose.

Proposition [TR Proposition 4.7] (The check is exact for the recorded facts). At any point in a run, the recorded fact set $\{\mathrm{rank}(\rho) \geq \mathrm{rank}(\rho') + k\}$ is satisfiable over $\mathbb{N}$ iff G has no positive-weight cycle. Hence the guard fails exactly when the facts recorded so far are jointly unsatisfiable.

*(Standard difference-constraints/Bellman–Ford duality — worth stating because it delimits what the check decides: unsatisfiability of the **recorded** facts, a sound under-approximation of rank-unsatisfiability of the configuration. Whether enough facts are always recorded in time is exactly TR Lemma 5.5.)*

5. Metatheory of solving

Lemma [TR Lemma 5.1] (Solution preservation, with a policy coordinate). Write $\widehat{C}$ for the constraint set $\Phi \cup \mathrm{eqns}(\sigma) \cup \mathrm{constr}(B)$ denoted by a configuration ($\mathrm{eqns}(\sigma) = \{x \approx \sigma(x)\}$; $\mathrm{constr}(B)$ the stored bounds/adjacencies as constraints), and consider $\mathrm{Sol}(\widehat{C})$ under TR Definition 3.4. The rules above divide:

Semantic steps — all dimension rules, all row-inequality rules (RI-cap, RI-closed-op, RI-short-closed, RI-deficit), and the checking content of the equality rules (the flat zip of RE-closed, the outer-edge dimension equalities, the rank-overflow failure of RE-open-closed, the discard of RE-same-empty, the occurs failure of RE-same-occurs, and the entailed flat content of RE-open-closed and RE-nested bindings) — preserve $\mathrm{Sol}(\widehat{C})$ exactly, up to extension of γ to freshly introduced variables (RI-deficit): every solution of the old configuration extends to one of the new, and every solution of the new restricts to one of the old. fail steps of this kind are taken only when $\mathrm{Sol}(\widehat{C}) = \emptyset$.

Policy steps — the **placement** component of the row-variable bindings in RE-open-closed and RE-nested, and the **deferred-equation resolutions** at closing for RE-cross and RE-same-rot (both word-equation residues resolved by the upward close) — *refine* Sol: the bound variable's flat content is entailed, but the committed split selects one of the pairwise $\sqsubseteq$-incomparable representatives of TR Proposition 2.9(ii), and substituting the committed value into stored row-subtyping constraints may strengthen them when the variable appears on the upper/operand side. Every solution of the new configuration restricts to a solution of

the old; conversely a solution of the old survives into the new after re-placing the bound variables' markers, which is possible iff no row-subtyping constraint of $\widehat{C}$ discriminates the placement — the witness in the marker-placement remark above is the counterexample, and the marker-provenance discussion in TR §6.1 is its operational trace. A fail arising from a substituted placement, where another representative would have passed, is therefore a *policy rejection*, not semantic unsatisfiability.

Definition [TR Definition 5.2] (Rank abstraction). For a configuration with constraint denotation $\widehat{C}$, its *rank abstraction* $\lfloor \widehat{C} \rfloor$ is the finite set of rank facts read off one constraint at a time: a row constraint relating $l \cdot \langle \rho \rangle \cdot r$ to $l' \cdot \langle \rho' \rangle \cdot r'$ (as result/operand of $\preceq$, resp. as an equality) contributes $\mathrm{rank}(\rho) \geq \mathrm{rank}(\rho') + (|l'| + |r'|) - (|l| + |r|)$ (both directions for an equality); open-vs-closed pairs contribute the corresponding absolute bounds on $\mathrm{rank}(\rho)$. A *rank model* is an assignment $r : \mathrm{RowVars} \to \mathbb{N}$ satisfying $\lfloor \widehat{C} \rfloor$. Every $\gamma \in \mathrm{Sol}(\widehat{C})$ induces a rank model $\rho \mapsto \mathrm{rank}(\gamma\rho)$ by TR Definition 2.12; the converse fails — the abstraction forgets dimensions and bases.

Lemma [TR Lemma 5.3] (Rank conservativity). Every non-fail rule maps a configuration with a rank model to one with a rank model agreeing on the old variables (extended on fresh ones). RI-deficit extends by $r(\rho') := r(\rho_{\mathrm{res}}) - k \geq 0$, justified by the triggering constraint's own rank fact; everything else preserves the model verbatim. *(As the TR notes, the previously drafted RE-cross fresh-variable binding would break this lemma — its* $\mathrm{rank}(\rho_2) \geq |s|$ *commitment is not implied by the consumed fact — an independent reason for RE-cross's correction to deferral.)* □

Theorem [TR Theorem 5.4; TR Lemma 5.5] (Termination). Let Φ_0 be finite.

(a) If Φ_0 has a rank model — in particular whenever $\mathrm{Sol}(\Phi_0) \neq \emptyset$ — then every fair run terminates, with or without the cycle check; moreover the check never fires on such inputs (its recorded facts are entailed, hence satisfied by the rank model, hence positive-cycle-free by TR Proposition 4.7), so guarding is semantically free.

(b) If Φ_0 has no rank model, then $\mathrm{Sol}(\Phi_0) = \emptyset$ and no run reaches solved form (TR Proposition 5.7 plus TR Lemma 5.1 would otherwise produce a solution, hence a rank model); every run either fails or diverges, and the cycle check is what converts divergence into failure. The proof (completed in the TR) is built around:

Detection Lemma [TR Lemma 5.5]. Every infinite run inserts, at some finite stage, an edge closing a positive-weight cycle in G. (Hence, with the guard of TR Definition 4.6, infinite runs do not exist, and rank-unsatisfiable inputs fail in finite time.)

Definition [TR Definition 5.6] (Solved form). A final configuration $\langle \emptyset; \sigma_\star; B_\star \rangle$ is in *solved form*: $\sigma_\star$ idempotent; every variable in $B_\star$ unsolved; each dimension variable carries at most one atom lower bound; all stored row caps and adjacencies are between unsolved variables and contain no pending pins (else a rule would fire).

Proposition [TR Proposition 5.7] (The residual store is always satisfiable; $\gamma_\uparrow$ is its greatest solution). Define $\gamma_\uparrow$ on the unsolved variables: $\gamma_\uparrow(\alpha) = 1_\emptyset$, $\gamma_\uparrow(\rho) = [\,] \cdot \diamond \cdot [\,]$, extended through $\sigma_\star$ on solved variables ($\gamma_\uparrow(x) = \gamma_\uparrow(\sigma_\star(x))$). Then (i) $\gamma_\uparrow \in \mathrm{Sol}(\mathrm{constr}(B_\star) \cup \mathrm{eqns}(\sigma_\star))$; (ii) $\gamma_\uparrow$ is the pointwise-greatest element of that solution set: for every solution γ and every variable x, $\gamma(x) \sqsubseteq \gamma_\uparrow(x)$ at dimension sort and $\gamma(x) \preceq \gamma_\uparrow(x)$ at row sort.

(Scope notes. First: solved form here is taken with the RE-same-rot deferrals discharged — an in-flight deferred equation $x \cdot t = s \cdot x$ is satisfied by $\gamma_\uparrow$ iff $s = t$, so a surviving $s \neq t$ deferral postpones (i) to the closing step that resolves or fails it; the success direction of TR Corollary 5.8 is unaffected, since an end-to-end successful run has discharged its deferrals. Second: (ii) compares rows by $\preceq$ *of necessity:* $\mathrm{eqns}(\sigma_\star)$ *is read under* $\approx$*, so a solution may re-place a solved row variable's marker, and distinct placements are* $\sqsubseteq$*-incomparable (TR Proposition 2.9(ii)) — when the solver substitutes, it commits one placement, a policy incompleteness rather than order-theoretic strength. The greatestness is over the solution set of the* final configuration*; it does not by itself give greatestness over* $\mathrm{Sol}(\Phi_0)$ *— that transfer is the content, and the caveat, of TR Proposition 6.4.)*

Corollary [TR Corollary 5.8] (Decision). A fair run fails iff the *policy-strengthened* system is unsatisfiable — Φ_0 plus TR Lemma 5.1's policy commitments (placements from RE-open-closed and RE-nested, and deferred-equation resolutions from RE-cross and RE-same-rot). Success implies $\mathrm{Sol}(\Phi_0) \neq \emptyset$ outright (TR Proposition

5.7 composed back through the preservation chain — the success direction is unconditional). Failure implies $\mathrm{Sol}(\Phi_0) = \emptyset$ *when the failing step is semantic*; a policy rejection can fail an $\approx$-satisfiable input (the witness in the marker-placement remark above). On inputs satisfiable *and* placement-undiscriminated, runs terminate by TR Theorem 5.4 and cannot fail, so they decide positively. On unsatisfiable inputs, runs cannot succeed; that they *terminate* — making the solver a decision procedure rather than a semi-decision procedure — is exactly TR Lemma 5.5.

Theorem [TR Theorem 5.9] (Soundness, representation, most generality). Let a fair run on finite Φ_0 terminate in $\langle \emptyset; \sigma_\star; B_\star \rangle$. Then, with $V = \mathrm{vars}(\Phi_0)$:

1. *(Representation)* $\gamma \in \mathrm{Sol}(\Phi_0)$ iff γ extends to a ground $\hat{\gamma}$ (on the fresh variables) with $\hat{\gamma} = \hat{\gamma} \circ \sigma_\star$ and $\hat{\gamma} \in \mathrm{Sol}(\mathrm{constr}(B_\star))$.
2. *(No guessing / entailment)* every binding $x \mapsto T$ in $\sigma_\star$ is conservative: every solution of Φ_0 extends to one satisfying $x = T$.
3. *(Most generality among models)* for every substitution $\sigma \models \Phi_0$ there is u with $u \circ \sigma_\star = \sigma$ on V — indeed $u = \sigma$ works: $\sigma = \sigma \circ \sigma_\star$ on V.

Caveat to the whole theorem (the $\approx$/policy weakening, with TR Definition 3.4's adoption): (1)'s "iff" and (2)'s "every solution extends" hold exactly for runs of semantic steps; across TR Lemma 5.1's *policy steps* the forward direction holds after re-placing the policy-committed markers, hence unconditionally only on placement-undiscriminated inputs (the conjectured surface fragment — TR Corollary 5.8). (3) is up to $\approx$ as the proof now states: $\sigma_\star$ is most general in content; its marker placements are the policy's, not canonical. This is the theorem-level form of the marker-placement caveat in TR §6.1.

Caveat to (1)/(2): the extension over fresh variables is existential, exactly as in standard unification with fresh-variable introduction; the answer is unique up to renaming of fresh variables.

Corollary [TR Theorem 5.9(2)] (Forced variables). $\sigma_\star$ binds only variables whose values (relative to the remaining free ones) are entailed by Φ_0 — by TR Theorem 5.9(2). Conversely every variable left unsolved genuinely admits at least two solutions or carries only its top default ($\gamma_\uparrow$ vs. e.g. committing a cap), by TR Proposition 5.7 and inspection of solved form.

Corollary [TR Corollary 5.10] (Principal model, recovered). If $B_\star$ stores no atom caps and no row caps (adjacencies allowed), then $\sigma_\star$ (read as a model, free variables universal) satisfies $\sigma_\star \models \Phi_0$, and by TR Theorem 5.9(3) it is the $\leq$-least model: a principal model exists exactly in this case. TR Example 3.6 shows the restriction is necessary.

6. Closing

Definition [TR Definition 6.1] (Closing policy). Variables carry a provenance tag: *leaf* (belonging to a terminal tensor's shape; a subset are *parameters*) or *interior*. Closing runs after solving, so dimension leaves close to the glb defined by the saturated store of TR Definition 4.1. Closing then proceeds interleaved: 1. *(Close leaves, downward.)* For each unsolved leaf variable: bind a dimension variable to its stored atom cap if any, else to $1_\emptyset$ — except a **parameter** with no cap at some position: error ("unspecified hidden dimension"). Bind a row variable to the join (TR Proposition 2.7) of the *ground parts* of its stored caps, with *holes* (positions where no cap is ground) filled by $1_\emptyset$, and no-further-axes beyond the caps' extent; parameters error at holes. 2. *(Re-solve.)* The bindings re-emit the variables' stored constraints (TR Definition 4.1); run the solver to a new solved form. 3. *(Close interiors, upward.)* Bind every remaining variable to its top ($1_\emptyset$ / the empty row); re-solve once more (re-emissions against tops discharge by DI-top and TR Proposition 2.6, so this cannot fail).

The downward leaf choice is local: with fully ground caps it is the join of the stored lower bounds, i.e. the $\sqsubseteq$-least value satisfying those caps alone. A row cap with holes is a policy guess, since filling a hole with $1_\emptyset$ is generally incomparable with filling it by a concrete atom; parameter errors are the guard against making that guess silently.

Proposition [TR Proposition 6.2] (Closing terminates and is sound). The interleave terminates, and if no error/fail occurs, the resulting total ground substitution $\gamma_{\mathrm{close}} \in \mathrm{Sol}(\Phi_0)$. Ground commitments add

no variable-to-variable rank facts, so the variable–variable fragment keeps its model and bounds all growth (the deficit potential consults only open-result lineages); a rank-breaking commitment therefore fails *finitely*, through the absolute fragment. TR Lemma 5.5 is not needed here.

Remark [TR Definition 4.1; TR Example 6.3] (Transitive glb **and incomplete leaf closing).** Let

$$\Phi_1 = \{3_b \sqsubseteq \alpha,\ \alpha \sqsubseteq \beta,\ 5_b \sqsubseteq \beta\}$$

with α, β leaves. By TR Definition 4.1, $3_b \sqsubseteq \alpha \sqsubseteq \beta$ contributes the lower bound $3_b \sqsubseteq \beta$, so $\mathrm{glb}(\beta) = 3_b \vee 5_b = 1_\emptyset$ by DI-cap. Thus solving collapses β to $1_\emptyset$ before leaf closing commits anything. The solutions of Φ_1 are exactly $\beta = 1_\emptyset$, $\alpha \in \{3_b, 1_\emptyset\}$, and deterministic closing selects $\alpha = 3_b, \beta = 1_\emptyset$ in every tested emission order; the branch $\beta \mapsto 5_b$ is not a legal close-to-glb run.

This order-independence is not completeness. Let

$$\Phi_2 = \{3_b \sqsubseteq \alpha,\ 5_b \sqsubseteq \beta,\ \gamma \sqsubseteq \alpha,\ \gamma \sqsubseteq \beta\}$$

with α, β leaves and γ interior. The store is satisfiable, e.g. $\alpha = \beta = 1_\emptyset, \gamma = 3_b$. Leaf-downward closing commits the capped leaves to their saturated GLBs, $\alpha \mapsto 3_b$ and $\beta \mapsto 5_b$; re-solving then pins γ below two distinct atoms and fails. This is a policy rejection: satisfying the store requires relaxing at least one capped leaf upward to $1_\emptyset$, and the deterministic closing policy deliberately does not backtrack to larger leaf shapes.

Proposition [TR Proposition 6.4] (Uniform-upward closing yields the greatest solution). $\gamma_\uparrow$ of TR Proposition 5.7 is a solution of Φ_0 — unconditionally: the soundness direction of TR Lemma 5.1 composes through policy steps. Its *greatestness* needs more care than the original one-line proof ("TR Proposition 5.7 plus TR Theorem 5.9(1)") admitted: that route uses TR Theorem 5.9(1)'s only-if direction, which policy steps break — the run's policy commitments strengthen the configuration (placements through substituted occurrences on operand sides, rank choices through deferred-equation resolutions), so TR Proposition 5.7(ii) quantifies over a possibly *proper* subset of $\mathrm{Sol}(\Phi_0)$. Three correct forms:

(i) *Final-configuration form, and why* $\preceq$. $\gamma_\uparrow$ is pointwise-greatest among the solutions of the final configuration, rows compared by $\preceq$ (TR Proposition 5.7). No marked-order ($\sqsubseteq$) strengthening holds at row sort: row equations — including $\mathrm{eqns}(\sigma_\star)$ — are read under $\approx$, so a solution may re-place the marker of a solved row variable, and distinct placements are pairwise $\sqsubseteq$-incomparable (TR Proposition 2.9(ii)); once a row variable is solved to a row of positive rank, no $\sqsubseteq$-greatest solution exists at all. The solver's substitution commits one placement — a policy incompleteness (TR Lemma 5.1), not order-theoretic strength.

(ii) *The unqualified marked claim is false.* $\Phi_0 = \{\langle\rho\rangle \approx [\,] \cdot \diamond \cdot [3,5]\}$: the run commits $\rho \mapsto [\,] \cdot \diamond \cdot [3,5]$ (inherit-the-split), so $\gamma_\uparrow(\rho) = [\,] \cdot \diamond \cdot [3,5]$, while $\gamma(\rho) = [3] \cdot \diamond \cdot [5] \in \mathrm{Sol}(\Phi_0)$ under TR Definition 3.4's $\approx$-semantics is $\sqsubseteq$-*incomparable* to it (TR Proposition 2.9(ii)).

(iii) *Absolute up to* $\approx$*, relative to the rank policy.* For every $\gamma \in \mathrm{Sol}(\Phi_0^{\mathrm{rk}})$ — Φ_0 plus the run's deferred-equation resolutions, i.e. the rank-level policy commitments of RE-cross and RE-same-rot — and every $x \in V$: $\gamma(x) \preceq \gamma_\uparrow(x)$ at row sort, plain $\sqsubseteq$ at dimension sort. **(proved in the TR)** Placement commitments need no exclusion: $\preceq$ is marker-blind on its lower side, which is exactly why the row subtype/refinement relation is the right comparison. Rank commitments do: a solution realizing a foreclosed sporadic/conjugate branch can have smaller rank at a variable than $\gamma_\uparrow$'s committed value, and no marker-blind order repairs a rank gap. On deferral-free runs $\Phi_0^{\mathrm{rk}} = \Phi_0$ and the statement is unconditional.

Remark [TR Example 3.6; §6] (Non-principality witnessed). $\Phi = \{3_b \sqsubseteq \alpha\}$, α a leaf: $\gamma_{\mathrm{close}}(\alpha) = 3_b$; $\gamma_\uparrow(\alpha) = 1_\emptyset$; both are solutions, neither factors through the other by substitution (TR Example 3.6). Closing selects by policy, justified by allocation (leaves) and parsimony (interiors), not by entailment.

7. Projection inference

Throughout, fix one operation; its shapes are solved and **closed** (ground). Its constraint set Φ_{op} is re-derived with **freshened projection identifiers**: each axis of each participating tensor occurrence carries an id

$p \in P$, injectively, used by this operation only; $\mathrm{sz}(p) \in \mathbb{N}_{\geq 1}$ is the axis's solved size. (The spec's fresh row variables are eliminated by the local re-solve — TR Lemma 7.7 — and its label variables resolve to operand axes, inheriting ids.)

Definition [TR Definition 7.1] (Projection language). Atoms $q ::= \mathsf{Proj}(p) \mid \mathsf{Sol}(idx)$, where idx is an externally supplied index: $\mathsf{Fix}(c)$ or an external symbol. Equations $e ::= \mathsf{Eq}(q_1, q_2) \mid \mathsf{Iter}(q)$, with Eq symmetric.

Definition [TR Definition 7.2] (Elaboration $[\![\cdot]\!]$). From the ground Φ_{op}:

- P1. $d_1 = d_2 \rightsquigarrow \mathsf{Eq}([\![d_1]\!], [\![d_2]\!])$.
- P2. $d_{\mathrm{res}} \sqsubseteq d_{\mathrm{op}}$ with $\mathrm{sz}(d_{\mathrm{op}}) = 1 \rightsquigarrow \mathsf{Eq}([\![d_{\mathrm{op}}]\!], \mathsf{Sol}(\mathsf{Fix}\,0))$ (sever and pin).
- P3. $d_{\mathrm{res}} \sqsubseteq d_{\mathrm{op}}$ otherwise $\rightsquigarrow \mathsf{Eq}([\![d_{\mathrm{res}}]\!], [\![d_{\mathrm{op}}]\!])$.
- P4. $R_1 \approx R_2$: outer-edge alignment; P1 per aligned pair (ranks match — shapes are closed and the equality held).
- P5. $R_{\mathrm{res}} \preceq R_{\mathrm{op}}$: outer-edge alignment; P2/P3 per aligned pair, with P2's severance applied row-wise (result side of the pair $\rightsquigarrow \mathsf{Iter}$, operand pinned); each surplus interior result axis $\rightsquigarrow \mathsf{Iter}([\![\cdot]\!])$.
- P6. each terminal axis $\rightsquigarrow \mathsf{Iter}([\![\cdot]\!])$. Side condition: $\mathsf{Sol}(\mathsf{Fix}\,c)$ is emitted only with $0 \leq c <$ the axis's size (slices are validated against solved sizes).

Definition [TR Definition 7.3] (Solver). A single pass over the finite equation set E maintaining: a union–find partition of P; a partial pin map on classes; an iterate set I of classes.

- $\mathsf{Eq}(\mathsf{Proj}\,p, \mathsf{Proj}\,q)$: union, fail if class sizes differ.
- $\mathsf{Eq}(\mathsf{Proj}\,p, \mathsf{Sol}\,i)$: pin p's class at i, fail on a conflicting pin.
- $\mathsf{Eq}(\mathsf{Sol}\,i, \mathsf{Sol}\,j)$: check $i = j$.
- $\mathsf{Iter}(\mathsf{Proj}\,p)$: add class to I.
- $\mathsf{Iter}(\mathsf{Sol}\,\_)$: no-op.

Labeling: pinned class $\mapsto$ its pin (pins dominate I); unpinned class in I with size $> 1 \mapsto$ a fresh iterator symbol; otherwise $\mapsto \mathsf{Fix}(0)$. The **product space** $P^{\times} = \prod_j R_j$, one factor $R_j = \{0..n_j{-}1\}$ per fresh iterator with n_j the class size; the **index map** π_T of tensor T maps each axis to its class's label.

Theorem [TR Theorem 7.4] (Canonicity). For a finite E: the partition, the pin map, the iterate set, and the labeling are independent of processing order; the solver fails iff (a) some $\approx$-class (the least equivalence containing the $\mathsf{Eq}(\mathsf{Proj}, \mathsf{Proj})$ pairs) contains two ids of different sizes, or (b) some class receives two distinct pins, or (c) some $\mathsf{Eq}(\mathsf{Sol}\,i, \mathsf{Sol}\,j)$ has $i \neq j$. On success the output is unique up to a bijective renaming of the fresh iterator symbols.

Definition [TR Definition 7.5] (Reduction; coverage). A factor j of $P^{\times}$ is a *reduction axis* iff no component of π_{LHS} is $\mathsf{Iter}(s_j)$.

Proposition [TR Proposition 7.6] (Injectivity, surjectivity, and the reduction characterization). With core labels only (Iter/Fix):

(i) $\pi_{\mathrm{LHS}} : P^{\times} \to \mathrm{Addr}$ is injective iff every product variable occurs in some component of π_{LHS} — i.e. iff there is no reduction axis.

(ii) π_{LHS} is surjective onto the LHS's index domain iff every LHS axis of size > 1 is labeled Iter and the map (axis $\mapsto$ its variable) is injective on those axes.

(iii) Hence: the accumulating read-modify-write is required iff a reduction axis exists (non-injectivity), and pre-initialization is required iff π_{LHS} is non-surjective or (under an erasing accumulator) non-injective — the `=:+` analysis, now a corollary.

Lemma [TR Lemma 7.7] (Local re-solve succeeds; locality). If the global solve and closing succeeded, then for each operation: (a) the per-operation re-derivation (spec template with fresh row/dimension variables against the ground operand shapes) has a solution, and the core solver finds it, eliminating every spec variable; (b) the resulting $\approx$ depends only on Φ_{op} — by construction of the freshened ids, no constraint of any other operation mentions any id of P, so no external fact can enter the closure.

Theorem [TR Theorem 7.8] (Soundness of projections w.r.t. shapes). Let shapes be solved and closed, the per-operation elaboration as in TR Definition 7.2 with its side condition, and the local solve succeed. Then: 1. *(Bounded addressing)* for every participating tensor T and every $p \in P^{\times}$, $\pi_T(p)$ is a valid multi-index: componentwise, an $\mathsf{Iter}(s_j)$ component addresses an axis whose size equals n_j, and $s_j < n_j$; a $\mathsf{Fix}(c)$ component has $c <$ the axis size. 2. *(Co-iteration is finer than size-equality)* $p \approx q \Rightarrow \mathrm{sz}(p) = \mathrm{sz}(q)$, and $\approx$ is the **least** equivalence justified by this operation's own constraints (TR Theorem 7.4 + TR Lemma 7.7(b)) — in particular, axes identified in size by other operations' constraints are not co-iterated here.